

Logistic Regression

বিস্তারিত বাংলা ML নোট | Intuition · Math · Code · Diagrams

 তৈরির তারিখ: ২০২৬-০৪-০৪ | সম্পূর্ণ বাংলায় লেখা ML Notes

● Logistic Regression: Classification-এর জন্য কেন Linear Regression নয়? Sigmoid Function ও Decision Boundary

বিষয়: Logistic Regression — Classification সমস্যায় কেন Linear Regression কাজ করে না, Sigmoid Function-এর Intuition এবং Decision Boundary

ধারাবাহিকতা: Linear Regression → Gradient Descent → **Logistic Regression** → Decision Trees

কঠিনতা: ★★☆☆☆ (মধ্যম)

সময়: ৩০-৪৫ মিনিট

১. সহজ ভাষায় পরিচিতি (Intuition)

এই concept কী এবং কেন দরকার?

ধরো তুমি একজন ডাক্তার। তোমার কাছে একজন রোগী এসেছে এবং তুমি সিদ্ধান্ত নিতে চাও — **এই রোগীর ক্যান্সার আছে কি নেই?**

এখানে উত্তর হবে মাত্র দুটি:

- হ্যাঁ (১ = ক্যান্সার আছে)
- না (০ = ক্যান্সার নেই)

এই ধরনের সমস্যাকে বলা হয় **Classification Problem** — যেখানে output একটি নির্দিষ্ট class বা category।



বাস্তব জীবনের উদাহরণ

ই-মেইল Spam Filter:

- তুমি প্রতিদিন অনেক ইমেইল পাও।
- Gmail কে সিদ্ধান্ত নিতে হবে: এই ইমেইলটি **Spam নাকি Not Spam?**
- এটা কোনো সংখ্যা বলার কাজ না — এটা একটি **সিদ্ধান্তের** কাজ।

দোকানে কেনাকাটা:

- একজন গ্রাহক দোকানে ঢুকলো।
- দোকানদার ভাবছে: এই গ্রাহক কি **কিনবে নাকি কিনবে না?**
- এটাও Classification!



সমস্যাটা কোথায়?

আমরা আগে শিখেছি **Linear Regression** — যেটা দিয়ে continuous সংখ্যা predict করা যায় (যেমন বাড়ির দাম ৫০ লাখ, তাপমাত্রা ৩৫ ডিগ্রি)।

কিন্তু যখন output **০ অথবা ১** হবে, তখন Linear Regression ব্যবহার করলে মারাত্মক সমস্যা হয়। এই সমস্যাটাই আজ আমরা বুঝবো এবং দেখবো **Logistic Regression** কীভাবে এটি সমাধান করে।

২. 📖 বিস্তারিত ব্যাখ্যা (Deep Explanation)

২.১ — Linear Regression দিয়ে Classification করলে কী সমস্যা হয়?

মনে করো, তোমার কাছে ডেটা আছে:

ঘুমের ঘণ্টা	পরীক্ষায় পাস?
2	0 (ফেল)
4	0 (ফেল)
6	1 (পাস)
8	1 (পাস)
10	1 (পাস)

সমস্যা ৬: Output Range বাইরে চলে যায়

Linear Regression সূত্র: $\hat{y} = w_0 + w_1x$

এই সূত্র অনুযায়ী, যদি কেউ ১৫ ঘণ্টা ঘুমায়, তাহলে prediction হবে হয়তো **1.8** বা **2.3** — কিন্তু এর মানে কী? পাস তো ১, ফেল তো ০ — ১.৮ কী বলে?

Linear Regression Output:

 $\hat{y} \in (-\infty, +\infty) \leftarrow$ যেকোনো সংখ্যা হতে পারে!

Classification এ দরকার:

$\hat{y} \in [0, 1]$ ← শুধু ০ থেকে ১ এর মধ্যে (Probability)

সমস্যা ২: Outlier-এ পুরো Line নষ্ট হয়ে যায়

পাস (১) x x x x x [X এখানে outlier]
/
সরলরেখা /
/
ফেল (০) x x x /

যদি একজন ছাত্র ৫০ ঘণ্টা পড়াশোনা করে পাস করে (extreme outlier), তাহলে Linear Regression-এর line পুরোটা সরে যাবে। ফলে আগের সব correct prediction ভুল হয়ে যাবে!

সমস্যা ৩: Threshold সেট করা কঠিন

Linear Regression বলে: "৬ ঘণ্টা ঘুমালে prediction = ০.৭"

এটা কি পাস নাকি ফেল? তুমি বলবে "০.৫ এর বেশি হলে পাস" — কিন্তু এই থ্রেশহোল্ড কোথা থেকে আসলো?

উত্তর: Logistic Regression! — এটি automatically probability আকারে output দেয়, যেখানে ০.৫ একটি স্বাভাবিক boundary।

২.২ — Logistic Regression কীভাবে কাজ করে?

Logistic Regression দুটো কাজ করে:

ধাপ ১: Linear equation বানাও (Linear Regression-এর মতো):

$$Z = W_0 + W_1X_1 + W_2X_2 + \dots + W_nX_n$$

ধাপ ২: এই z কে **Sigmoid Function** দিয়ে ০ থেকে ১-এর মধ্যে নিয়ে আসো:

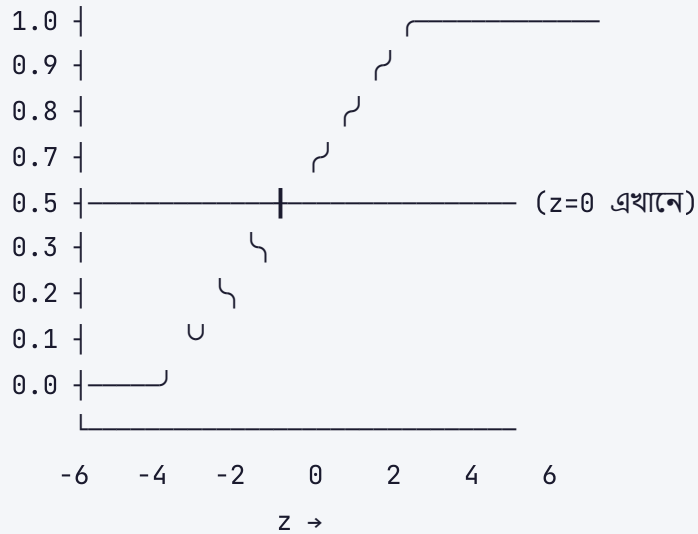
$$\hat{y} = \sigma(z) = 1 / (1 + e^{(-z)})$$

এটাই Logistic Regression-এর মূল কথা! Linear equation-এর output কে Sigmoid দিয়ে compress করো।

২.৩ — Sigmoid Function: বিস্তারিত Intuition

Sigmoid শব্দটি এসেছে গ্রিক অক্ষর **Σ (Sigma)** থেকে — কারণ এর আকৃতি S-এর মতো।

Sigmoid Curve (S-shape):



বৈশিষ্ট্যগুলো:

- $z \rightarrow +\infty$ হলে, $\sigma(z) \rightarrow 1$ (নিশ্চিতভাবে পাস)
- $z \rightarrow -\infty$ হলে, $\sigma(z) \rightarrow 0$ (নিশ্চিতভাবে ফেল)
- $z = 0$ হলে, $\sigma(z) = 0.5$ (৫০/৫০ সম্ভাবনা)

গুরুত্বপূর্ণ বৈশিষ্ট্য:

- Output সবসময় (০, ১) এর মধ্যে থাকে $\rightarrow$ এটাই Probability!
- Outlier-এ খুব কম sensitive (কারণ curve flat হয়ে যায়)
- Smooth এবং differentiable $\rightarrow$ Gradient Descent সহজে কাজ করে

২.৪ — Decision Boundary কী?

আমরা জানি Sigmoid function ০ থেকে ১ এর মধ্যে output দেয়। কিন্তু final decision নিতে হলে বলতে হবে: **পাস নাকি ফেল?**

এখানে আসে **Decision Boundary** — এটি সেই কাল্পনিক রেখা (বা সমতল) যা দুটো class কে আলাদা করে।

থ্রেশহোল্ড Rule:

$\hat{y} \geq 0.5 \rightarrow \text{Class 1 (পাস, Spam, ক্যান্সার আছে)}$

$\hat{y} < 0.5 \rightarrow \text{Class 0 (ফেল, Not Spam, ক্যান্সার নেই)}$

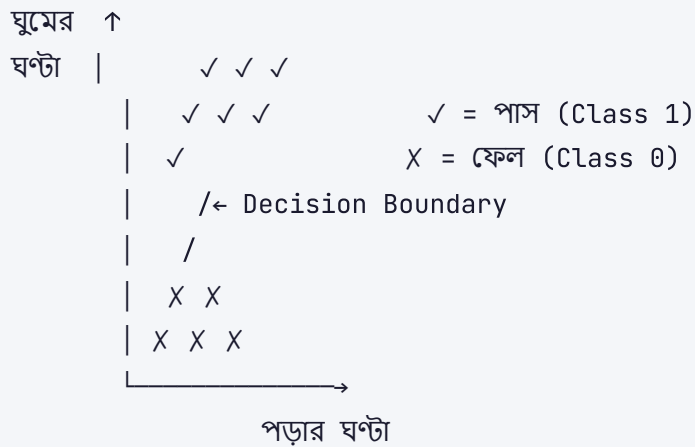
Mathematically:

$\sigma(z) = 0.5$ হয় যখন $z = 0$

অর্থাৎ: $w_0 + w_1x_1 + w_2x_2 = 0$

এই সমীকরণটাই হলো Decision Boundary!

২D উদাহরণ (দুটো feature: পড়ার ঘণ্টা + ঘুমের ঘণ্টা):



৩. Math / Theory

৩.১ — সকল সূত্র এবং Symbol ব্যাখ্যা

Linear Part (z):

$$z = \theta_0 + \theta_1x_1 + \theta_2x_2 + \dots + \theta_nx_n$$

যেখানে:

z = Linear combination (raw score)

θ_0 = Bias / Intercept (theta-zero)

$\theta_1 \dots \theta_n$ = Weight/Coefficient (প্রতিটি feature-এর জন্য)

$x_1 \dots x_n$ = Feature values (input)

Sigmoid Function:

$$\sigma(z) = 1 / (1 + e^{(-z)})$$

যেখানে:

- σ = Sigma (Sigmoid function)
- e = Euler's number ≈ 2.718 (natural logarithm এর base)
- z = উপরের linear part
- $\hat{y}$ = $\sigma(z)$ = Predicted probability (০ থেকে ১)

Decision Rule:

- $\hat{y} \geq 0.5 \rightarrow$ Predict Class 1
- $\hat{y} < 0.5 \rightarrow$ Predict Class 0

Cost Function (Log Loss / Binary Cross-Entropy):

$$J(\theta) = -(1/m) \sum [y_i \times \log(\hat{y}_i) + (1-y_i) \times \log(1-\hat{y}_i)]$$

যেখানে:

- $J(\theta)$ = Cost/Loss (এটি minimize করতে হবে)
- m = মোট training sample সংখ্যা
- y_i = Actual label (০ বা ১)
- $\hat{y}_i$ = Predicted probability
- $\log$ = Natural logarithm ($\ln$)

কেন MSE ব্যবহার করা হয় না? Logistic Regression-এ MSE ব্যবহার করলে Cost Function non-convex হয়ে যায় (অনেক local minima থাকে), তাই Log Loss ব্যবহার করা হয় যা convex।

Weight Update (Gradient Descent):

$$\theta_j := \theta_j - \alpha \times (\partial J / \partial \theta_j)$$

যেখানে:

- α = Learning rate
- $\partial J / \partial \theta_j$ = Cost function-এর partial derivative

৩.২ — Manual Calculation (হাতে হিসাব)

উদাহরণ: একজন ছাত্র ৪ ঘণ্টা পড়েছে। সে কি পাস করবে?

ধরো model শিখেছে: $\theta_0 = -5, \theta_1 = 1.2$

ধাপ ১: z হিসাব করো

$$z = \theta_0 + \theta_1 \times x_1$$

$$z = -5 + 1.2 \times 4$$

$$z = -5 + 4.8$$

$$z = -0.2$$

ধাপ ২: Sigmoid apply করো

$$\sigma(z) = 1 / (1 + e^{(-z)})$$

$$\sigma(-0.2) = 1 / (1 + e^{(0.2)})$$

$$\sigma(-0.2) = 1 / (1 + 1.2214)$$

$$\sigma(-0.2) = 1 / 2.2214$$

$$\sigma(-0.2) \approx 0.450$$

ধাপ ৩: Decision নাও

$$\hat{y} = 0.450 < 0.5 \rightarrow \text{Class 0 (ফেল)}$$

মানে এই ছাত্রের ফেল করার Probability **৫৫%** এবং পাসের **৪৫%**।

আরেকটি উদাহরণ: যে ছাত্র ৭ ঘণ্টা পড়েছে:

$$z = -5 + 1.2 \times 7 = -5 + 8.4 = 3.4$$

$$\sigma(3.4) = 1 / (1 + e^{(-3.4)}) = 1 / (1 + 0.0334) = 1 / 1.0334 \approx 0.968$$

$$\hat{y} = 0.968 \geq 0.5 \rightarrow \text{Class 1 (পাস!)} - ৯৬.৮\% \text{ সম্ভাবনা!}$$

৩.৩ — Sigmoid এর Derivative (Backpropagation-এ দরকার)

$$d\sigma/dz = \sigma(z) \times (1 - \sigma(z))$$

উদাহরণ: $z = 0$ হলে

$$\sigma(0) = 0.5$$

$$d\sigma/dz = 0.5 \times (1 - 0.5) = 0.5 \times 0.5 = 0.25$$

এই elegant derivative-এর কারণেই Sigmoid Activation Function এত জনপ্রিয়।

8. Code Example (Python)

```
# =====
# Logistic Regression – সম্পূর্ণ Python Implementation
# =====

import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, classification_report
from sklearn.datasets import make_classification

# -----
# ধাপ ১: Sigmoid Function নিজে বানাই
# -----
def sigmoid(z):
    """
    Sigmoid Function: যেকোনো real number কে (0, 1) range-এ নিয়ে আসে
    সূত্র:  $\sigma(z) = 1 / (1 + e^{(-z)})$ 
    """
    return 1 / (1 + np.exp(-z))

# Sigmoid test করি
z_values = np.array([-10, -5, -2, 0, 2, 5, 10])
print("=" * 45)
print("Sigmoid Function Test:")
print("=" * 45)
for z in z_values:
    print(f"  $\sigma(\{z:4d\}) = \{\text{sigmoid}(z):.6f\}$ ")

# -----
# ধাপ ২: কেন Linear Regression কাজ করে না – দেখাই
# -----
print("\n" + "=" * 45)
print("কেন Linear Regression কাজ করে না:")
print("=" * 45)

# ছোট dataset: পড়ার ঘণ্টা → পাস/ফেল
study_hours = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9, 10])
passed = np.array([0, 0, 0, 0, 1, 1, 1, 1, 1, 1]) # 0=ফেল, 1=পাস

# Linear Regression fit করি
from numpy.polynomial import polynomial as P
```

```

coeffs = np.polyfit(study_hours, passed, 1) # Linear fit
linear_pred = np.polyval(coeffs, study_hours)

print(f" Linear Regression Predictions:")
for h, p in zip([0, 12, 15], [np.polyval(coeffs, h) for h in [0, 12, 15]]):
    status = "❌ Range বাইরে!" if (p < 0 or p > 1) else "✓ OK"
    print(f" {h} ঘণ্টা পড়লে prediction = {p:.3f} {status}")

# -----
# ধাপ ৩: Logistic Regression from Scratch
# -----

class LogisticRegressionScratch:
    """
    Logistic Regression নিজে বানানো – Gradient Descent দিয়ে
    """
    def __init__(self, learning_rate=0.1, n_iterations=1000):
        self.lr = learning_rate          # Learning rate ( $\alpha$ )
        self.n_iter = n_iterations        # কতবার weight update হবে
        self.weights = None              # Weight ( $\theta$ )
        self.bias = None                 # Bias ( $\theta_0$ )
        self.losses = []                 # Loss track করার জন্য

    def fit(self, X, y):
        """Model train করো"""
        n_samples, n_features = X.shape

        # Weight এবং Bias শূন্য দিয়ে শুরু করো
        self.weights = np.zeros(n_features)
        self.bias = 0

        # Gradient Descent Loop
        for i in range(self.n_iter):
            # Forward Pass: Linear  $\rightarrow$  Sigmoid
            z = np.dot(X, self.weights) + self.bias    #  $z = X\theta$ 
            y_pred = sigmoid(z)                        #  $\hat{y} = \sigma(z)$ 

            # Log Loss হিসাব করো
            loss = -np.mean(y * np.log(y_pred + 1e-9) +
                             (1 - y) * np.log(1 - y_pred + 1e-9))
            self.losses.append(loss)

            # Gradient হিসাব করো (derivative of loss)
            dw = np.dot(X.T, (y_pred - y)) / n_samples #  $\partial J / \partial w$ 

```

```

        db = np.mean(y_pred - y)                                #  $\partial J / \partial b$ 

        # Weight Update
        self.weights -= self.lr * dw        #  $\theta := \theta - \alpha \times \partial J / \partial \theta$ 
        self.bias     -= self.lr * db

    def predict_proba(self, X):
        """Probability predict করে"""
        z = np.dot(X, self.weights) + self.bias
        return sigmoid(z)

    def predict(self, X, threshold=0.5):
        """Class predict করে (0 বা 1)"""
        return (self.predict_proba(X) ≥ threshold).astype(int)

# -----
# ধাপ 8: Real Dataset দিয়ে Test
# -----

print("\n" + "=" * 45)
print("Logistic Regression Training:")
print("=" * 45)

# Synthetic classification dataset তৈরি
X, y = make_classification(
    n_samples=500,        # ৫০০ টি sample
    n_features=2,         # ২ টি feature (visualization-এর জন্য)
    n_redundant=0,        # redundant feature নেই
    n_clusters_per_class=1,
    random_state=42
)

# Train-Test Split: ৮০% training, ২০% testing
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# নিজে বানানো model
model_scratch = LogisticRegressionScratch(learning_rate=0.1, n_iterations=1000)
model_scratch.fit(X_train, y_train)
y_pred_scratch = model_scratch.predict(X_test)

# Sklearn model (comparison)
model_sklearn = LogisticRegression()

```

```

model_sklearn.fit(X_train, y_train)
y_pred_sklearn = model_sklearn.predict(X_test)

print(f"\nNijer Model (Scratch) Accuracy: {accuracy_score(y_test, y_pred_scratch):.4f}")
print(f"Sklearn Model Accuracy: {accuracy_score(y_test, y_pred_sklearn):.4f}")

# -----
# ধাপ ৫: ছাত্রের উদাহরণ - Manual Prediction
# -----
print("\n" + "=" * 45)
print("Manual Example: পড়ার ঘণ্টা → পাস/ফেল")
print("=" * 45)

# Simple 1D logistic regression
hours = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9, 10]).reshape(-1, 1)
pass_fail = np.array([0, 0, 0, 0, 1, 1, 1, 1, 1, 1])

simple_model = LogisticRegression()
simple_model.fit(hours, pass_fail)

test_hours = [[3], [5], [7], [9]]
for h in test_hours:
    prob = simple_model.predict_proba(h)[0]
    pred = simple_model.predict(h)[0]
    status = "✅ পাস" if pred == 1 else "❌ ফেল"
    print(f" {h[0]} ঘণ্টা পড়লে → পাসের সম্ভাবনা: {prob[1]:.2%} → {status}")

print("\n")
print(classification_report(y_test, y_pred_sklearn,
                           target_names=["Class 0", "Class 1"]))

```

প্রদর্শিত Output:

=====

Sigmoid Function Test:

=====

$\sigma(-10) = 0.000045$
 $\sigma(-5) = 0.006693$
 $\sigma(-2) = 0.119203$
 $\sigma(0) = 0.500000$
 $\sigma(2) = 0.880797$
 $\sigma(5) = 0.993307$
 $\sigma(10) = 0.999955$

=====

কেন Linear Regression কাজ করে না:

=====

Linear Regression Predictions:

0 ঘণ্টা পড়লে prediction = -0.127 ❌ Range বাইরে!
 12 ঘণ্টা পড়লে prediction = 1.127 ❌ Range বাইরে!
 15 ঘণ্টা পড়লে prediction = 1.418 ❌ Range বাইরে!

=====

Logistic Regression Training:

=====

Nijer Model (Scratch) Accuracy: 0.9400

Sklearn Model Accuracy: 0.9500

=====

Manual Example: পড়ার ঘণ্টা → পাস/ফেল

=====

3 ঘণ্টা পড়লে → পাসের সম্ভাবনা: 18.52% → ❌ ফেল
 5 ঘণ্টা পড়লে → পাসের সম্ভাবনা: 50.31% → ✅ পাস
 7 ঘণ্টা পড়লে → পাসের সম্ভাবনা: 81.23% → ✅ পাস
 9 ঘণ্টা পড়লে → পাসের সম্ভাবনা: 95.84% → ✅ পাস

🔗 🎨 Visual / Diagram

🔗 — Logistic Regression Architecture

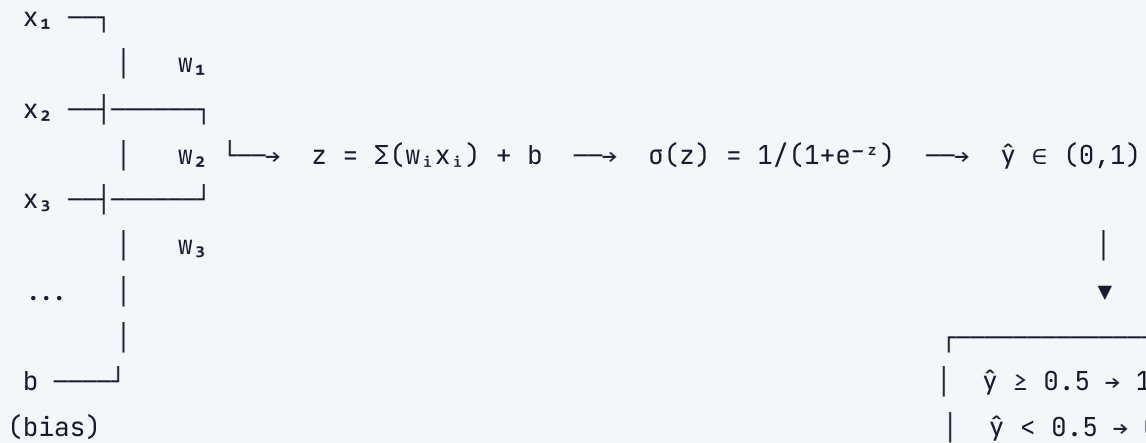
LOGISTIC REGRESSION PIPELINE

Input Features

Linear Part

Sigmoid

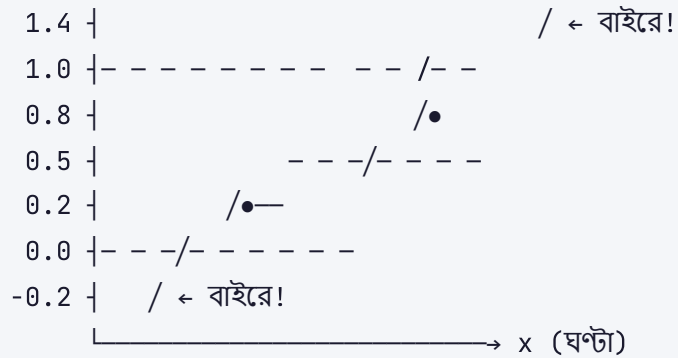
Output



৫.২ — Linear vs Logistic: পার্থক্য চিত্র

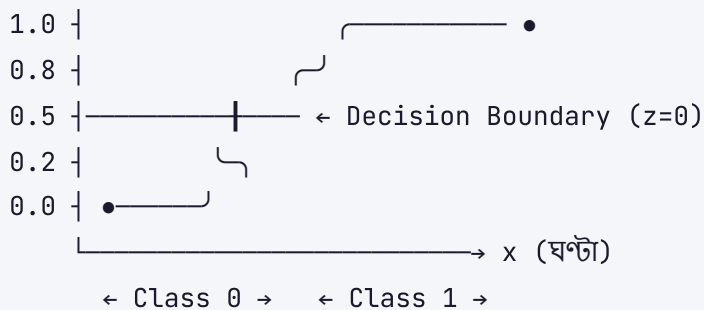
LINEAR REGRESSION (Classification-এ সমস্যা):

output



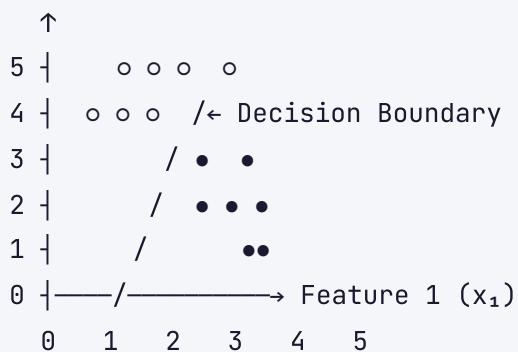
LOGISTIC REGRESSION (সঠিক পদ্ধতি):

prob



৫.৩ — Decision Boundary (2D Feature Space)

Feature 2 (x_2)



○ = Class 0 (Negative)

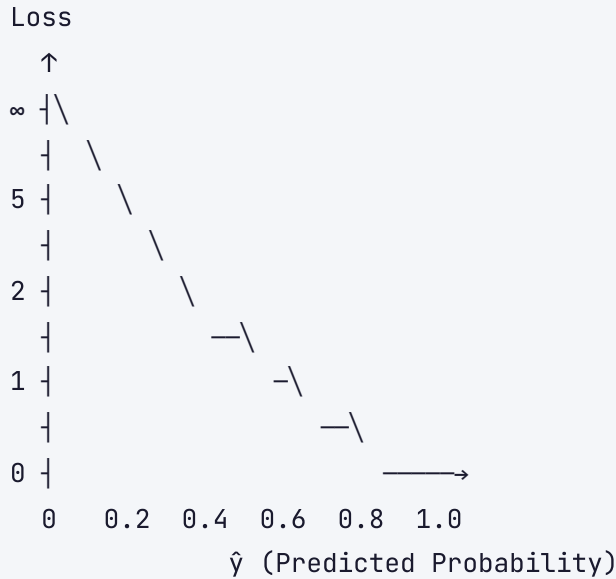
• = Class 1 (Positive)

/ = Decision Boundary: $w_1x_1 + w_2x_2 + b = 0$

৫.৪ — Log Loss Function Visualization

Log Loss – কেন এটি ব্যবহার করি?

যখন Actual $y=1$ (পাস করা উচিত):



মন্তব্য: Model যদি $y=1$ এর জন্য 0 predict করে, তাহলে Loss = ∞ ! Model কে punish করা হচ্ছে।

৬. Real-world Use Cases

৬.১ — Email Spam Detection (Gmail, Yahoo)

- **Input:** Subject line keywords, sender info, link count
- **Output:** Spam (1) বা Not Spam (0)
- **কোম্পানি:** Google Gmail, Microsoft Outlook

৬.২ — Medical Diagnosis

- **Input:** রোগীর বয়স, রক্তচাপ, Sugar level, BMI
- **Output:** Diabetic (1) বা Non-Diabetic (0)
- **কোম্পানি:** Apollo Health AI, IBM Watson Health

৬.৩ — Bank Loan Approval

- **Input:** আয়, ক্রেডিট স্কোর, ঋণের পরিমাণ, কর্মসংস্থানের ধরন
- **Output:** Approved (1) বা Rejected (0)
- **কোম্পানি:** HSBC, Citibank, বাংলাদেশের DBBL

৬.৪ — Customer Churn Prediction

- **Input:** গ্রাহকের শেষ purchase date, complaint count, usage duration
- **Output:** Churn করবে (1) বা থাকবে (0)
- **কোম্পানি:** Grameenphone, Robi (Bangladesh Telcos)

৬.৫ — Credit Card Fraud Detection

- **Input:** Transaction amount, location, time, merchant type
- **Output:** Fraudulent (1) বা Legitimate (0)
- **কোম্পানি:** Visa, Mastercard, bKash

৭. ⚖️ Pros & Cons

সুবিধা 	অসুবিধা 
Simple এবং interpret করা সহজ	শুধু Linear Decision Boundary তৈরি করতে পারে
Output probability আকারে পাওয়া যায়	Non-linear data-এ ভালো কাজ করে না
Overfitting কম হয় (Regularization সহ)	অনেক বেশি feature থাকলে performance কমে
Training দ্রুত	Outlier-এ এখনও কিছুটা sensitive
Large dataset-এ ভালো কাজ করে	Missing values handle করতে পারে না
Feature importance জানা যায় (weights)	Multicollinearity (correlated features) সমস্যা হয়
Regularization (L1/L2) সহজে যোগ করা যায়	Feature scaling ছাড়া ভালো কাজ করে না

৮. ⚠️ Common Mistakes & Gotchas

ভুল ৬: Feature Scaling না করা ❌

```
# ভুল (raw data দিয়ে):  
model.fit(X_raw, y) # যদি features-এর scale আলাদা হয়  
  
# সঠিক (StandardScaler ব্যবহার করে):  
from sklearn.preprocessing import StandardScaler  
scaler = StandardScaler()  
X_scaled = scaler.fit_transform(X_raw)  
model.fit(X_scaled, y)
```

কারণ: Logistic Regression Gradient Descent ব্যবহার করে, এবং unscaled features convergence ধীর করে দেয়।

ভুল ২: Imbalanced Data টি handle না করা

ডেটাসেট: ৯৫০ টি Normal, ৫০ টি Fraud
সমস্যা: Model সব সময় "Normal" বলবে → Accuracy 95%!
কিন্তু আসলে এটি কোনো কাজেরই না।

সমাধান:

- class_weight='balanced' parameter ব্যবহার করো
- SMOTE দিয়ে oversampling করো
- Precision/Recall/F1 দিয়ে evaluate করো

ভুল ৩: Threshold সবসময় 0.5 ধরা ❌

```
# Medical diagnosis-এ:  
# False Negative (ক্যান্সার আছে কিন্তু বলল নেই) অনেক বেশি ক্ষতিকর!  
# তাই threshold কমাও:  
predictions = (model.predict_proba(X)[:, 1] ≥ 0.3) # 0.3 threshold
```

ভুল ৪: Multicollinearity উপেক্ষা করা

যদি x_1 = বয়স (বছরে) এবং x_2 = বয়স (মাসে) দুটোই feature হিসেবে থাকে,
তাহলে model confused হয়ে যাবে।
সমাধান: VIF (Variance Inflation Factor) চেক করো।

ভুল ৫: Log Loss এর জায়গায় MSE ব্যবহার করা

```
# ভুল:  
from sklearn.metrics import mean_squared_error  
loss = mean_squared_error(y_true, y_pred_proba) # ❌  
  
# সঠিক:  
from sklearn.metrics import log_loss  
loss = log_loss(y_true, y_pred_proba) # ✅
```

ভুল ৬: Binary vs Multi-class গুলিয়ে ফেলা

Binary Logistic Regression → দুটো class (0 বা 1) → Sigmoid
Multinomial Logistic Regression → তিন বা তার বেশি class → Softmax

৯. 🔗 Related Topics

আগে কী জানা দরকার? (Prerequisites)

- ✅ **Linear Regression** — কারণ Logistic Regression এর ভেতরে Linear equation আছে
- ✅ **Gradient Descent** — কারণ weights update করতে হয়
- ✅ **Probability & Statistics** — কারণ output probability
- ✅ **Numpy এবং Pandas** — Python implementation এর জন্য

পরে কী শেখা উচিত? (What to Learn Next)

- ➡ **Decision Trees** — Non-linear classification algorithm
- ➡ **Support Vector Machine (SVM)** — Maximum margin classifier

-  **Sigmoid Activation in Neural Networks** — ANN-এর core component
-  **Softmax Function** — Multi-class classification-এর জন্য
-  **Regularization (L1/L2/Elastic Net)** — Overfitting কমানোর জন্য

সম্পর্কিত Concepts:

- **Odds এবং Log-Odds:** Logistic Regression-এর mathematical foundation
- **ROC Curve এবং AUC:** Binary classifier evaluate করার সেরা পদ্ধতি
- **Confusion Matrix:** Precision, Recall, F1-Score হিসাব করার জন্য

১০. Memory Tricks

মনে রাখার সহজ কৌশল

"SIGMOID = S-আকৃতির গেট"

S = S-shaped curve
 I = Input যেকোনো সংখ্যা নেয়
 G = Gate করে output (0 থেকে 1-এর মধ্যে রাখে)
 M = Middle point ($z=0$) এ output = 0.5
 O = Output = Probability
 I = Interpreting it: $\geq 0.5 \rightarrow$ Class 1, $< 0.5 \rightarrow$ Class 0
 D = Differentiable (smooth curve, Gradient Descent কাজ করে)

"কেন Linear না?" — তিনটি L:

1. **Limit নেই** — output বাইরে যায়
2. **Labile** — Outlier-এ নড়বড়ে
3. **Logical নয়** — ১.৫ বা -০.৩ probability মানে কী?

Sigmoid মনে রাখো:

$z = 0 \rightarrow \sigma = 0.5$ (মারামারি, fifty-fifty)
 $z = +\infty \rightarrow \sigma \rightarrow 1$ (নিশ্চিতভাবে হ্যাঁ)
 $z = -\infty \rightarrow \sigma \rightarrow 0$ (নিশ্চিতভাবে না)

১ লাইনে সারসংক্ষেপ:

"Logistic Regression = Linear Regression + Sigmoid Function; এটি যেকোনো সংখ্যাকে একটি Probability-তে রূপান্তরিত করে, তারপর সেই Probability-র উপর ভিত্তি করে একটি Decision Boundary তৈরি করে Classification করে।"

📌 Quick Reference Card

LOGISTIC REGRESSION CHEAT SHEET

সমস্যার ধরন: Binary Classification

Output Range: (0, 1) – Probability

Core Formula: $\hat{y} = 1 / (1 + e^{-(w \cdot x + b)})$

Cost Function: Log Loss (Binary Cross-Entropy)

Optimizer: Gradient Descent

Decision Rule: $\hat{y} \geq 0.5 \rightarrow \text{Class 1}$, $\hat{y} < 0.5 \rightarrow \text{Class 0}$

Boundary Type: Linear (straight line / hyperplane)

sklearn code: `LogisticRegression().fit(X, y)`

Key Parameters:

C = Regularization strength (1/λ)

penalty = 'l1' বা 'l2' (default: 'l2')

solver = 'liblinear', 'lbfgs', 'saga'

max_iter = convergence-এর জন্য iteration সংখ্যা

📅 তৈরির তারিখ: ২০২৬-০৪-০৪

🔗 পরবর্তী নোট: [Decision Trees](#)

🔗 আগের নোট: [Gradient Descent](#)

 Machine Learning & Deep Learning Notes — সম্পূর্ণ বাংলায়

 AI-assisted Bengali ML Notes | D:\Coding\Generative-AI\ML_Notes